

nvidia-nccl介绍

📘 本页面由边俊柯于2022/06/17迁移自wiki 陈庆澍空间

1 简介

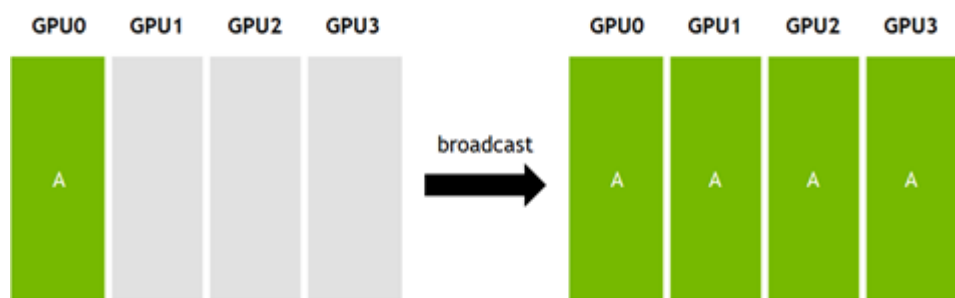
NCCL（发音为“Nickel”）是一个多GPU集群通信原语库，它具有拓扑感知能力，可以轻松集成到您的应用程序中。NCCL最初作为一个开源研究项目开发，因此NCCL的特点为轻量级，仅依赖于通常的C++和CUDA库。NCCL可以部署在单进程或多进程应用程序中，透明地处理进程间通信。最后，NCCL的API格式与MPI collective的API格式基本相同，如果你有MPI collective经验，那可以很快上手NCCL。

2 Collective Communication

并行任务的通信一般可以分为Point-to-point communication和Collective communication。P2P通信这种模式只有一个sender和一个receiver，实现起来比较简单。第二种Collective communication包含多个sender多个receiver，一般的通信原语包括broadcast, gather, all-gather, scatter, reduce, all-reduce, reduce-scatter等。

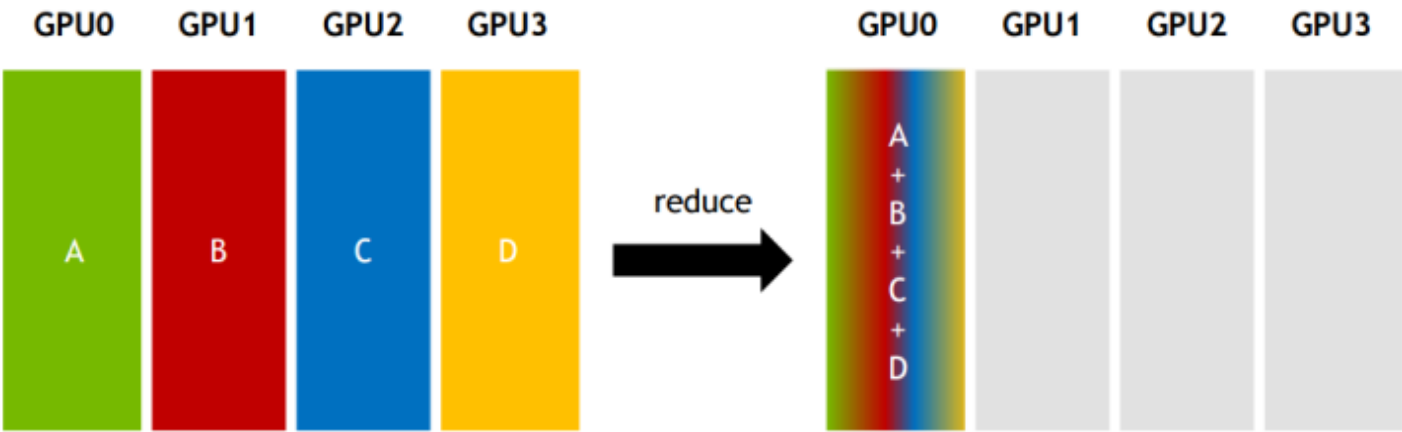
2.1 broadcast

一个sender的数据，发送给所有节点



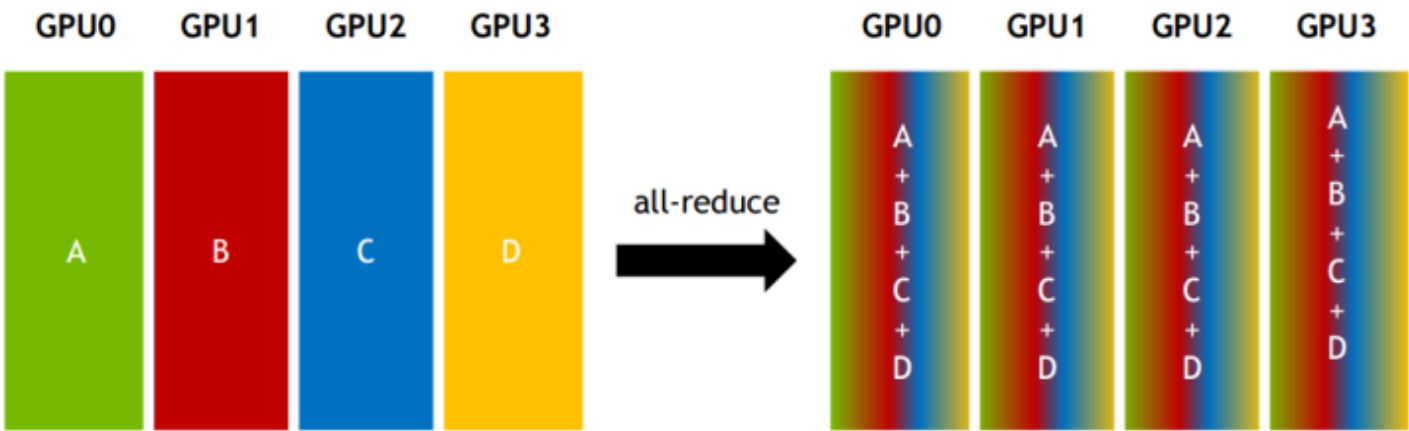
2.2 reduce

从多个sender那里接收数据，最终combine到一个节点上。



2.3 all-reduce

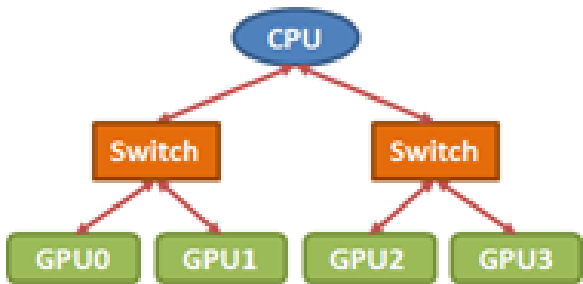
从多个sender那里接收数据，最终combine到每一个节点上。



3 网络拓扑&ring-base collectives

3.1 网络拓扑

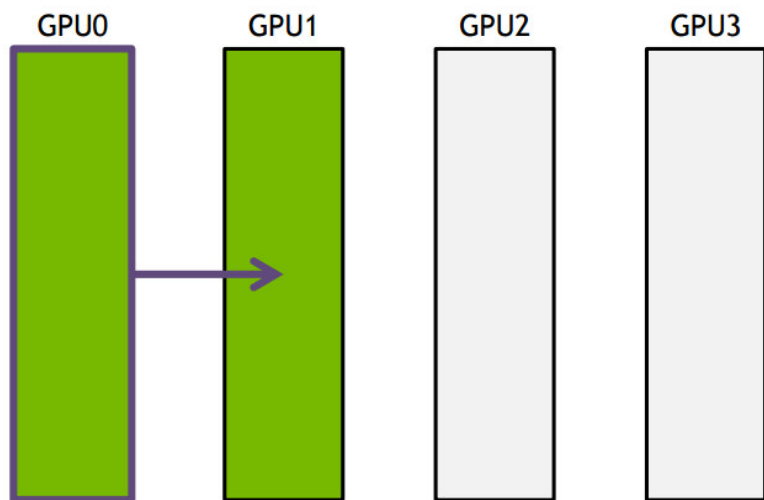
单机多卡之间有不同的拓扑结构，影响着数据之间的传输效率。



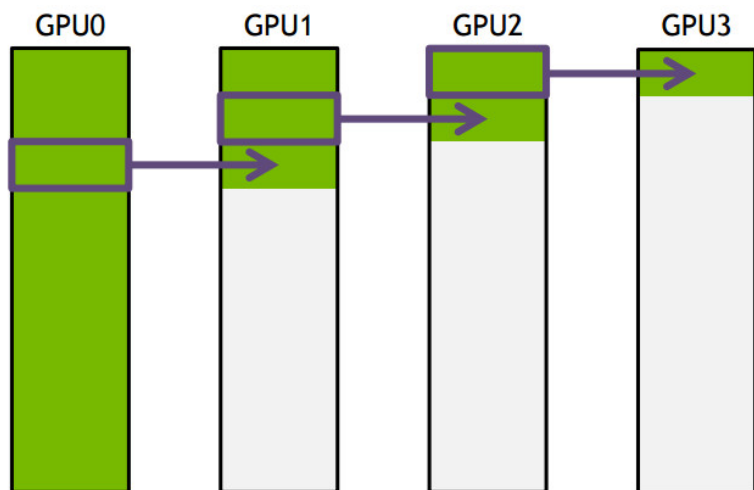
上图展示了一种常见的拓扑结构。以GPU0想向其他卡broadcast数据为例，数据传输先从GPU0 to GPU2，然后GPU0 to GPU1，GPU2 to GPU3的效率比先GPU0 to GPU1，然后GPU 0 to GPU 2，GPU 1 to GPU 3效率更高，因为后者的第二部需要竞争CPU。

3.2 ring-base collectives

ring-base collectives将所有的通信节点通过首尾连接形成一个单向环，数据在环上依次传输。以broadcast为例， 假设有4个GPU，GPU0为sender将信息发送给剩下的GPU，按照环的方式依次传输，GPU0-->GPU1-->GPU2-->GPU3。



若数据量为N，带宽为B，整个传输时间为 $(K-1) N/B$ 。时间随着节点数线性增长，不是很高效。下面把要传输的数据分成S份，每次只传N/S的数据量，传输过程如下所示：

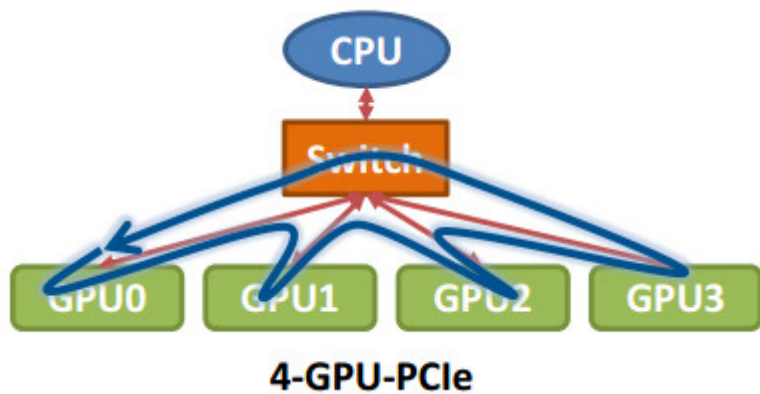


GPU1接收到GPU0的一份数据后，也接着传到环的下个节点，这样以此类推，最后花的时间为

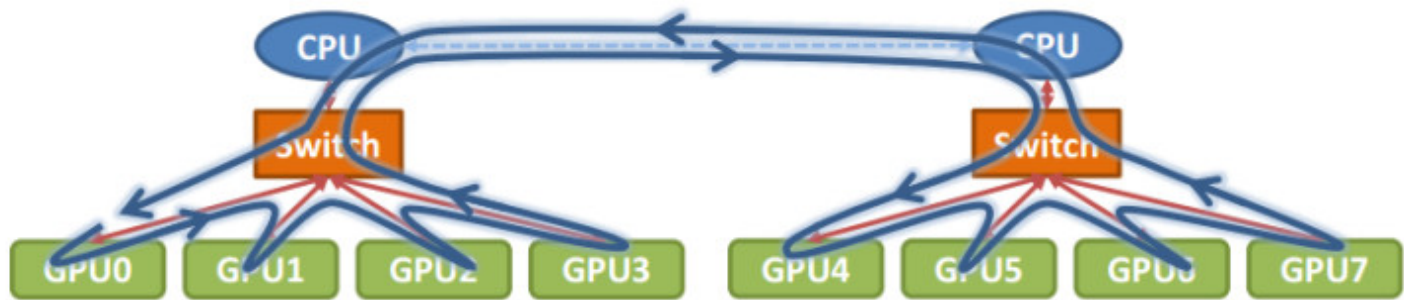
$S * (N/S/B) + (k-2) * (N/S/B) = N(S+K-2)/(SB) \rightarrow N/B$ ，条件是S远大于K，即数据的份数大于节点数，这个很容易满足。所以通信时间不随节点数的增加而增加，只和数据总量以及带宽有关。

环形是一种很好的方法来优化broadcast，且即使当应用于“树形”PCIe拓扑时，环算法也为几乎所有的标准集合操作提供接近最优的带宽。但请注意，**选择正确的振铃顺序仍然很重要。**

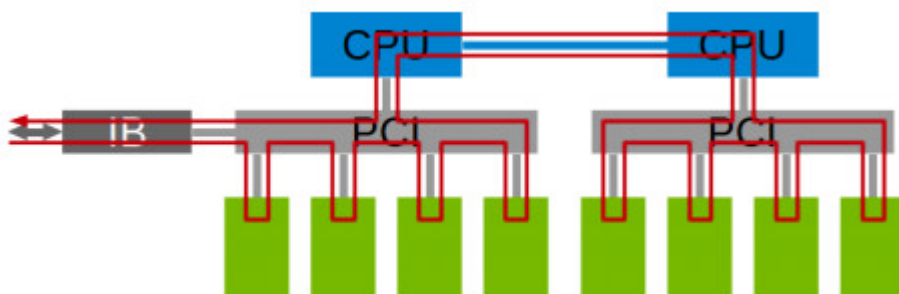
单机4卡通过同一个PCIe switch挂载在一棵CPU的场景：



单机8卡通过两个CPU下不同的PCle switch挂载的场景：



多机场景下，也是构成一个环



4 NCCL实现

NCCL实现成CUDA C++ kernels。NCCL 1.0版本支持单机多卡，卡之间通过PCle、NVlink等通讯。NCCL 2.0支持多级多卡，多机间通过Sockets、InfiniBand等通讯。

在实现上，NCCL的特性包括：

1. NCCL内collectives的实现方式为ring-style，以提供最大带宽。
2. NCCL能够自动对GPU进行编号并实现最佳ring顺序。
3. 许多collectives实现时需要一个缓冲区来存储中间结果。为了最小化每个GPU上的几MB的内存开销，NCCL将大型集合分成许多小块。对于集合算法的每个步骤和块。
4. 启动单独的内核和cudaMemcpy调用是非常低效的。对于一个collectives算法，将每一个步骤实现为一个kernel，每一个chunk数据都执行cudaMemcpy，这种做法是非常低效的。因此，NCCL内的每一个collective操作都由一个大kernel实现。

5. NCCL广泛使用GPUDirect peer to peer直接访问方式在处理器之间传输数据。